MacBasic

User's Guide and Language Reference

Structured BASIC for macOS and GNUstep

Version 0.2 | July 2026

MacBasic Project

About This Manual

This manual describes the MacBasic implementation contained in this project. MacBasic is a structured, line-number-free BASIC with optional compatibility syntax inspired by Microsoft AmigaBASIC. It runs as a document-oriented AppKit application on macOS and is organized to build with GNUstep.

> **Compatibility** Portable behavior takes precedence over emulating Amiga hardware. Commands involving memory, screens, speech, menus, sound voices, and animated objects use host-safe abstractions. Explicit limitations are identified in the reference.

Conventions

Form	Meaning
UPPERCASE	A keyword typed as shown; keywords are case-insensitive.
lowercase	A value, variable, name, or expression supplied by the programmer.
[optional]	An optional syntax element.
a | b	Choose one of the alternatives.
Monospaced block	Program text or output.

Examples use UTF-8 source files with the .bas filename extension. Screen coordinates use points. Canvas coordinates begin at the top-left of the view. String literals must be delimited by the straight ASCII double quote character (", U+0022). Curly typographic quotation marks are not string delimiters.

Contents

CHAPTER 1

Welcome to MacBasic

A modern, structured BASIC with classic personal-computing immediacy

What MacBasic Is

MacBasic combines an approachable BASIC syntax with structured procedures, document-based editing, native windows, retained drawing views, sound, processes, menus, events, and portable compatibility features. Programs do not require line numbers. Named labels and numeric line labels are available when porting older programs.

A First Program

```
SUB Greet(name$)
  PRINT "Hello, " + name$
END SUB

Greet("Mac")
```

Choose Run in the document window. Text written by PRINT appears in the white output pane below the editor.

A First Window

```
WINDOW OPEN 1, "Hello", 520, 300, 120, 160
VIEW ADD 10, 1, 10, 10, 500, 260
CLEAR 10, "#F5F3FF"
DRAW TEXT 10, "Hello from MacBasic", 80, 100, "#312E81"
```

Design Goals

- No mandatory line numbers.
- Readable block structure with SUB, FUNCTION, IF, WHILE, and FOR.
- Native document windows and host integration.
- A familiar compatibility layer for classic AmigaBASIC programs.
- Safe portable replacements for hardware-specific operations.

CHAPTER 2

Using the IDE

Documents, editing, running, and output

Document Windows

Each .bas file is an NSDocument. Opening several programs creates independent editor windows, interpreters, output panes, and unsaved-change states. The title bar displays the document filename. New documents begin with a tutorial program that demonstrates procedures and drawing.

Editor

The source editor uses a parchment background and live syntax highlighting. Keywords, strings, numbers, labels, procedure calls, and comments receive distinct colors. Highlighting is display-only; saved files remain plain UTF-8 text.

Editing Commands

Command	Shortcut	Action
New	Command-N	Create a new untitled BASIC document.
Open	Command-O	Open a .bas document.
Save	Command-S	Save the current document.
Cut / Copy / Paste	Command-X/C/V	Use the standard first-responder editing operations.
Undo / Redo	Command-Z / Shift-Command-Z	Undo or redo text edits.
Run	Button	Run a snapshot of the current source.
Stop	Button	Request that the running interpreter stop.

Command-Line Mode

```
./build/MacBasic program.bas
```

When a source path is passed on the command line, MacBasic runs without opening the editor. PRINT writes to standard output. Native windows and drawing views are ignored with a diagnostic, and BEEP writes the terminal bell.

Compiling a Program

Choose File > Compile or use the Compile button, then select Application or Tool. An application is packaged as a native macOS app wrapper or GNUstep app wrapper and opens with the MacBasic window environment. A tool is a standalone terminal executable. When building an application, choose a custom icon or use the generic MacBasic icon. Custom macOS icons should be .icns files; GNUstep accepts .png, .tiff, or .icns files. Both targets contain the MacBasic runtime and an embedded copy of the source, so the original .bas file is not needed at run time. Build on the operating system where the result will run. The optional --console argument selects headless execution, in which native window operations are ignored.

```
./build/MacBasic --compile-tool program.bas program
./program

./build/MacBasic --compile-app program.bas Program.app
./build/MacBasic --compile-app program.bas Program.app MyProgram.icns
```

Building MacBasic

```
make test
make app
open build/MacBasic.app
```

On GNUstep, source the GNUstep environment before running make. The GNUstep GUI and development packages must already be installed.

CHAPTER 3

Language Fundamentals

Source layout, names, values, expressions, and comments

Program Lines

One statement normally occupies one source line. Colon-separated multiple statements are not supported in this version. Apostrophes and REM begin comments.

```
' An apostrophe comment
REM A traditional comment
answer = 42
```

Variables and Types

Variables are created by assignment. Names are case-insensitive. A suffix can request a compatibility type.

Suffix	Meaning	Current representation
$	String	NSString
%	Short-style integer	Host integer
&	Long-style integer	Host integer
!	Single-style numeric	Host double
#	Double-style numeric	Host double

> Note DEFINT, DEFLNG, DEFSNG, DEFDBL, and DEFSTR set the default type for unsuffixed names beginning with the declared letters. The declaration applies to subsequent assignments, arrays, input, loops, procedure parameters, and function results. Explicit suffixes always take precedence.

Constants and Expressions

Class	Operators
Arithmetic	+ - * / \ ^ MOD
Comparison	= <> < > <= >=
Logical	NOT AND OR XOR EQV IMP
String	+ concatenates strings

Comparisons return -1 for true and 0 for false. Logical operators work on integer values. Exponentiation binds more tightly than multiplication; comparisons are evaluated after arithmetic.

CHAPTER 4

Program Flow and Procedures

Decisions, loops, labels, subroutines, and functions

Structured IF

```
IF score >= 90 THEN
  PRINT "Excellent"
ELSEIF score >= 70 THEN
  PRINT "Passing"
ELSE
  PRINT "Try again"
END IF
```

Single-Line IF

```
IF ready THEN PRINT "Go" ELSE PRINT "Wait"
IF errorFlag THEN GOTO Handler
```

Restriction Embedded single-line actions use a restricted one-statement path. Prefer block IF for complex logic.

Loops

```
FOR i = 1 TO 10 STEP 2
  PRINT i
NEXT i

WHILE running
  SLEEP .05
WEND
```

EXIT FOR and EXIT WHILE leave the nearest matching loop.

Procedures

```
FUNCTION Area(w, h)
  RETURN w * h
END FUNCTION

SUB ShowArea(w, h)
  PRINT Area(w, h)
END SUB
```

Procedure arguments are copied into a local variable dictionary. Array objects remain shared references. SHARED, STATIC, COMMON, and DECLARE are accepted but currently have no scope-changing behavior.

Legacy Branching

```
10 GOSUB Report
20 GOTO Finished

Report:
```

```
PRINT "Working"
RETURN

Finished:
END
```

Numeric labels and named labels are optional compatibility features. A named label must appear on its own line.

CHAPTER 5

Arrays and Stored Data

Dimensioned values, DATA, READ, and RESTORE

Arrays

```
OPTION BASE 1
DIM scores(10), board(8, 8)
scores(1) = 95
board(4, 5) = 1
PRINT LBOUND(scores), UBOUND(scores)
ERASE board
```

Arrays support multiple dimensions. Each declared bound is inclusive. OPTION BASE may be 0 or 1 and affects arrays dimensioned afterward.

> **Bounds** An invalid array read currently returns zero and an invalid write is ignored. It does not yet raise a subscript error.

DATA Statements

```
Names:
DATA "Ada", "Grace", "Linus", 42
READ first$, second$, third$, answer
RESTORE Names
READ firstAgain$
```

DATA values form one sequential program-wide store. RESTORE without a label resets to the beginning. RESTORE label selects the first DATA statement after that label.

CHAPTER 6

Text and File Input/Output

Console interaction, formatting, and host files

PRINT

```
PRINT "Name"; ": "; name$
PRINT leftValue, rightValue
PRINT USING "###.##"; price
```

A semicolon joins output without padding. A comma advances to the next 14-character print zone. PRINT USING implements common numeric # fields; the complete Amiga formatting language is not yet available.

Interactive Input

```
INPUT "Your name"; name$
INPUT age
LINE INPUT "Message"; message$
```

The IDE displays a native input alert. Command-line mode reads from standard input.

Sequential Files

```
OPEN "report.txt" FOR OUTPUT AS #1
WRITE #1, "Ada", 42
PRINT #1, "plain text"
CLOSE #1

OPEN "report.txt" FOR INPUT AS #1
INPUT #1, name$, answer
CLOSE #1
```

Random Files

```
OPEN "records.dat" AS #2 LEN = 14
FIELD #2, 8 AS name$, 4 AS score$, 2 AS flags$
LSET name$ = "Ada"
LSET score$ = MKS$(98.5)
LSET flags$ = MKI$(1)
PUT #2, 1
GET #2, 1
CLOSE #2
```

Random records use fixed byte lengths and Latin-1 field strings. MKI$, MKL$, MKS$, and MKD$ create binary strings; CVI, CVL, CVS, and CVD recover numeric values.

Filesystem Statements

- FILES [path] lists a directory.
- CHDIR path changes the process working directory.

- KILL path removes a file.
- NAME old AS new renames or moves a file.
- SAVE path writes the interpreter's prepared source.
- CHAIN or RUN path loads and runs another program.

CHAPTER 7

Windows, Views, and Drawing

Native interfaces and retained canvas graphics

Opening Windows

```
WINDOW OPEN windowId, title$, width, height[, x, y]
```

```
WINDOW OPEN 1, "Chart", 640, 420, 160, 140
VIEW ADD 10, 1, 20, 20, 600, 360
```

Window and view IDs are positive integers used to route later commands; titles are display text only. The optional x and y values are screen coordinates. VIEW ADD places a canvas in the window selected by its ID.

Drawing Commands

Command	Principal arguments
CLEAR	viewId[, color]
DRAW LINE	viewId, x1, y1, x2, y2[, color]
DRAW RECT	viewId, x, y, width, height[, color[, fill]]
DRAW OVAL	viewId, x, y, width, height[, color[, fill]]
DRAW TEXT	viewId, text, x, y[, color]

```
CLEAR 10, "#F5F3FF"
DRAW RECT 10, 20, 20, 180, 90, "#4F46E5", 1
DRAW OVAL 10, 240, 20, 120, 120, "cyan", 0
DRAW TEXT 10, "MacBasic", 180, 190, "#312E81"
```

Compatibility Graphics

```
COLOR "#312E81"
PSET (10, 10)
LINE (20, 20)-(180, 90), "#4F46E5", B
CIRCLE (260, 100), 60, "cyan"
AREA (50, 200)
AREA (180, 260)
AREA (20, 280)
AREAFILL "#EC4899"
```

Portable graphics PALETTE, PATTERN, WIDTH, and SCROLL are accepted but currently have no rendering semantics. POINT returns zero. PAINT can fill a retained rectangle or oval containing the supplied point.

CHAPTER 8

Sound, Speech, and Processes

Host audio and operating-system integration

System and Named Sounds

```
BEEP
SOUND PLAY "Glass"
SOUND PLAY "/path/to/sound.wav"
SOUND STOP
```

Synthesized Tones

```
SOUND frequency, duration[, volume[, voice]]
```

```
WAVE 0, 0
SOUND 440, .25, .7, 0
WAVE 1, 1
SOUND 660, .25, 160, 1
```

Waveform 0 is sine, 1 is square, and 2 is sawtooth. Volume may be 0 through 1 or an Amiga-style value that is divided by 255. Duration is measured in seconds.

Speech

```
SAY "Welcome to MacBasic"
```

macOS uses the host say command. GNUstep builds attempt to use espeak when installed.

Processes

```
PROCESS RUN "/usr/bin/open", "https://www.gnu.org/software/gnustep/"
```

PROCESS RUN starts an external executable asynchronously with string arguments.

CHAPTER 9

Menus, Input, and Events

Polling and event-driven programs

Custom Menus

```
MENU 1, 0, 1, "Tools"
MENU 1, 1, 1, "Run Report"
ON MENU GOSUB MenuChosen
MENU ON
```

MENU(0) returns the last selected menu number and resets the menu selection. MENU(1) returns the last selected item.

Keyboard and Mouse

```
key$ = INKEY$
button = MOUSE(0)
x = MOUSE(1)
y = MOUSE(2)
```

INKEY$ consumes the last key recorded by the active canvas. MOUSE reports the left button and canvas coordinates. STICK and STRIG are reserved for a future controller bridge and currently return zero.

Event Traps

```
ON TIMER(1.0) GOSUB Tick
TIMER ON
ON MOUSE GOSUB Clicked
MOUSE ON
ON COLLISION GOSUB Hit
```

Events are checked between interpreted statements. They do not interrupt blocking input or SLEEP. Each event class retains one handler.

CHAPTER 10

Objects and Collision

Portable retained object state

Object Properties

```
OBJECT.X(1) = 20
OBJECT.Y(1) = 30
OBJECT.W(1) = 80
OBJECT.H(1) = 40
OBJECT.VX(1) = 2
OBJECT.AX(1) = .1
OBJECT.START 1
```

MacBasic stores portable object state by numeric identifier. X, Y, W, H, VX, VY, AX, and AY may be assigned and queried. START updates position and velocity between interpreted statements.

Collision

```
IF COLLISION(1, 2) THEN PRINT "Hit"
PRINT OBJECT.HIT(1, 2)
```

Collision uses axis-aligned bounding rectangles. Sprite masks, Amiga display planes, and blitter priorities are not emulated.

> **Rendering** OBJECT.SHAPE, OBJECT.CLIP, OBJECT.PLANES, and OBJECT.PRIORITY may be stored as compatibility properties but do not currently render or change layering.

CHAPTER 11

Errors, Memory, and System Control

Recovery and safe compatibility facilities

Error Handling

```
ON ERROR GOTO Handler
ERROR 42
PRINT "Continued"
END

Handler:
PRINT "Error"; ERR; " at line"; ERL
RESUME NEXT
```

ON ERROR GOTO installs one program-level handler. RESUME retries the failing location, RESUME NEXT continues with the following line, and RESUME label continues at a label.

Virtual Memory

```
POKE 100, 77
PRINT PEEK(100)
WAIT 100, 64
```

PEEK, PEEKW, PEEKL, POKE, POKEW, POKEL, and WAIT operate on a sandboxed dictionary. They never access process or hardware memory. VARPTR and SADD return host-related values and must not be treated as stable addresses.

Execution Control

- END and STOP finish the current program.
- SYSTEM and NEW finish execution; NEW does not clear the IDE document.
- SLEEP seconds pauses the interpreter.
- SWAP exchanges two variables.
- CLS clears the IDE output pane.
- LIST prints prepared source to output.

CHAPTER 12

Portability and Compatibility

What transfers between macOS, GNUstep, and AmigaBASIC

Portable Core

Variables, expressions, arrays, structured flow, procedures, DATA, ordinary files, retained drawing, and command-line execution are designed to behave consistently on Cocoa and GNUstep.

Host-Specific Services

Facility	macOS	GNUstep
Windows and views	AppKit/Cocoa	GNUstep GUI
Named sounds	NSSound	NSSound when supported
Speech	/usr/bin/say	/usr/bin/espeak
Document association	Info.plist and .icns	Desktop integration varies
Processes	NSTask	NSTask

Known Compatibility Limits

- One statement per source line; colon-separated statements are unsupported.
- DEF FN is not implemented; use a structured FUNCTION block instead.
- Several hardware-oriented keywords are accepted as no-ops.
- Amiga device names, serial channels, raw libraries, and 68k memory are not emulated.
- PRINT USING and graphics option syntax cover common rather than exhaustive forms.
- Unknown expression functions can currently evaluate to zero.

CHAPTER 13

Alphabetical Language Reference

Statements, operators, functions, and compatibility forms

Each entry documents the behavior of this MacBasic implementation. Syntax shown here takes precedence over historical AmigaBASIC syntax.

ABS

```
ABS(number)
```

Returns the absolute value of number.

Example

```
PRINT ABS(-12)
```

AND

```
left AND right
```

Returns the bitwise AND of two integer values.

Example

```
IF flags AND 4 THEN PRINT "set"
```

AREA

```
AREA (x, y)
```

Adds a point to the current polygon definition.

Example

```
AREA (10,10)
```

Implementation note Call AREAFILL after adding at least three points.

AREAFILL

```
AREAFILL [color]
```

Fills the polygon accumulated by AREA on the active canvas.

Example

```
AREAFILL "#4F46E5"
```

ASC

```
ASC(text$)
```

Returns the character value of the first character.

Example

```
PRINT ASC("A")
```

ATN

```
ATN(number)
```

Returns the arctangent in radians.

Example

```
angle = ATN(1)
```

BEEP

```
BEEP
```

Plays the host alert sound.

Example

```
BEEP
```

CHAIN

```
CHAIN path$
```

Loads and runs another source file.

Example

```
CHAIN "next.bas"
```

Implementation note Variables are not preserved through COMMON.

CHDIR

```
CHDIR path$
```

Changes the process working directory.

Example

```
CHDIR "/tmp"
```

CHR$

```
CHR$(code)
```

Returns the character corresponding to code.

Example

```
PRINT CHR$(65)
```

CIRCLE

```
CIRCLE (x, y), radius[, color]
```

Draws an outlined oval centered on x,y.

Example

```
CIRCLE (100,100), 50, "cyan"
```

Implementation note Arc and aspect parameters are not implemented.

CLEAR

```
CLEAR | CLEAR viewId[, color]
```

Without arguments, removes variables. With a view ID, clears that canvas.

Example

```
CLEAR 10, "white"
```

CLOSE

```
CLOSE [#channel] | WINDOW CLOSE windowId
```

Closes files or the native window selected by ID.

Example

```
CLOSE #1
```

CLS

```
CLS
```

Clears the IDE output pane.

Example

```
CLS
```

COLOR

```
COLOR color$
```

Sets the default compatibility drawing color.

Example

```
COLOR "#312E81"
```

COS

```
COS(number)
```

Returns the cosine of an angle in radians.

Example

```
x = COS(angle)
```

CVD / CVI / CVL / CVS

```
CVtype(binary$)
```

Decodes a binary field string created by the corresponding MK function.

Example

```
score = CVS(score$)
```

DATA

```
DATA constant[, constant...]
```

Adds numeric or quoted string constants to the program data store.

Example

```
DATA "Ada", 42
```

DATE$

```
DATE$
```

Returns the current date as MM-dd-yyyy.

Example

```
PRINT DATE$
```

DEFDBL / DEFINT / DEFLNG / DEFSNG / DEFSTR

```
DEFINT | DEFLNG | DEFSNG | DEFDBL | DEFSTR letter[-letter][, ...]
```

Sets the default type for unsuffixed names beginning with the selected letters. The type is enforced on assignments, arrays, READ and INPUT, FOR variables, procedure parameters, and function results. DEFINT and DEFLNG round values to the nearest integer; DEFSNG and DEFDBL select numeric values; DEFSTR selects strings.

Example

```
DEFINT A-C, I
DEFSTR N
average = 3.8
name = 42
```

Implementation note An explicit $, %, &, !, or # suffix overrides a letter-range declaration.

DIM

```
DIM name(bound[, bound...])[, ...]
```

Creates one or more multidimensional arrays.

Example

```
DIM board(8,8)
```

DRAW LINE

```
DRAW LINE viewId, x1, y1, x2, y2[, color]
```

Adds a retained line to the view selected by ID.

Example

```
DRAW LINE 10, 0,0,100,100, "red"
```

DRAW OVAL

```
DRAW OVAL viewId, x, y, w, h[, color[, fill]]
```

Adds a retained oval.

Example

```
DRAW OVAL 10, 20,20,80,80, "cyan", 1
```

DRAW RECT

```
DRAW RECT viewId, x, y, w, h[, color[, fill]]
```

Adds a retained rectangle.

Example

```
DRAW RECT 10, 20,20,80,80, "blue", 0
```

DRAW TEXT

```
DRAW TEXT viewId, text$, x, y[, color]
```

Draws host-system text in the view selected by ID.

Example

```
DRAW TEXT 10, "Hello", 20,20, "black"
```

END

```
END
```

Ends program execution.

Example

```
END
```

EOF

```
EOF(channel)
```

Returns -1 after all input lines have been consumed; otherwise 0.

Example

```
WHILE NOT EOF(1)
```

EQV

```
left EQV right
```

Returns the bitwise equivalence of integer operands.

Example

```
same = a EQV b
```

ERASE

```
ERASE array[, array...]
```

Removes dimensioned arrays.

Example

```
ERASE board
```

ERL

```
ERL
```

Within an error handler, contains the one-based failing source line.

Example

```
PRINT ERL
```

ERR

```
ERR
```

Within an error handler, contains the error number.

Example

```
PRINT ERR
```

ERROR

```
ERROR number
```

Raises a BASIC runtime error.

Example

```
ERROR 42
```

EXIT

```
EXIT FOR | EXIT WHILE
```

Leaves the nearest matching loop.

Example

```
IF done THEN EXIT FOR
```

EXP

```
EXP(number)
```

Returns e raised to number.

Example

```
PRINT EXP(1)
```

FIELD

```
FIELD #channel, length AS variable$[, ...]
```

Defines fixed fields for a random-access record.

Example

```
FIELD #2, 8 AS name$
```

FILES

```
FILES [path$]
```

Lists a host directory to output.

Example

```
FILES "."
```

FIX

```
FIX(number)
```

Truncates toward zero.

Example

```
PRINT FIX(-2.9)
```

FOR

```
FOR variable = start TO limit [STEP increment]
```

Begins a counted loop.

Example

```
FOR i=1 TO 10
```

FRE

```
FRE(value)
```

Returns a large host-capacity compatibility value.

Example

```
PRINT FRE(0)
```

Implementation note Not an Amiga memory measurement.

FUNCTION

```
FUNCTION name(parameters) ... END FUNCTION
```

Defines a value-returning procedure.

Example

```
FUNCTION Twice(n)
 RETURN n*2
END FUNCTION
```

GET

```
GET #channel[, record]
```

Reads a fixed random-access record into FIELD variables.

Example

```
GET #2, 1
```

GOSUB

```
GOSUB label
```

Branches to a legacy subroutine and remembers the return line.

Example

```
GOSUB Report
```

GOTO

```
GOTO label
```

Branches to a numeric or named label.

Example

```
GOTO Finished
```

HEX$

```
HEX$(number)
```

Returns uppercase hexadecimal text.

Example

```
PRINT HEX$(255)
```

IF

```
IF condition THEN ... [ELSEIF ...] [ELSE ...] END IF
```

Conditionally executes statements.

Example

```
IF x>0 THEN PRINT x
```

Implementation note Block form is recommended.

IMP

```
left IMP right
```

Returns bitwise implication.

Example

```
result = a IMP b
```

INKEY$

```
INKEY$
```

Returns and clears the active canvas's last key.

Example

```
key$ = INKEY$
```

INPUT

```
INPUT [prompt;] variable[, ...]
```

Reads values from a native alert or standard input.

Example

```
INPUT "Age"; age
```

INPUT$

```
INPUT$(count[, channel])
```

Returns up to count characters from input or a file line.

Example

```
key$ = INPUT$(1)
```

INSTR

```
INSTR([start,] text$, search$)
```

Returns the one-based search position or zero.

Example

```
PRINT INSTR("MacBasic","Basic")
```

INT

```
INT(number)
```

Returns the greatest integer not exceeding number.

Example

```
PRINT INT(2.9)
```

KILL

```
KILL path$
```

Deletes a host file.

Example

```
KILL "old.dat"
```

LBOUND

```
LBOUND(array)
```

Returns an array's lower bound.

Example

```
PRINT LBOUND(a)
```

LEFT$

```
LEFT$(text$, count)
```

Returns the leftmost characters.

Example

```
PRINT LEFT$("MacBasic",3)
```

LEN

```
LEN(value)
```

Returns the length of a string representation.

Example

```
PRINT LEN(name$)
```

LET

```
LET variable = expression
```

Assigns a value. LET is optional.

Example

```
LET total = 12
```

LINE

```
LINE (x1,y1)-(x2,y2)[, color[, B|BF]]
```

Draws a line, rectangle, or filled rectangle on the active canvas.

Example

```
LINE (0,0)-(100,80), "blue", B
```

LINE INPUT

```
LINE INPUT [prompt;] variable$ | LINE INPUT #channel, variable$
```

Reads one complete line.

Example

```
LINE INPUT message$
```

LIST / LLIST

```
LIST | LLIST
```

Writes prepared source to the output pane.

Example

```
LIST
```

Implementation note LLIST is not printer-specific.

LOC

```
LOC(channel)
```

Returns the current line or record index.

Example

```
PRINT LOC(1)
```

LOF

```
LOF(channel)
```

Returns file size when available.

Example

```
PRINT LOF(1)
```

LOG

```
LOG(number)
```

Returns the natural logarithm.

Example

```
PRINT LOG(10)
```

LSET / RSET

```
LSET field$ = value$ | RSET field$ = value$
```

Fits text to a random-file field with left or right padding.

Example

```
LSET name$ = "Ada"
```

MENU

```
MENU menu, item, state[, title$]
```

Creates or updates a native application menu item.

Example

```
MENU 1,1,1,"Run"
```

MENU()

```
MENU(0) | MENU(1)
```

Returns the last menu or item number.

Example

```
choice = MENU(1)
```

MID$

```
MID$(text$, start[, count])
```

Returns a substring using one-based indexing.

Example

```
PRINT MID$("MacBasic",4,5)
```

MKI$ / MKL$ / MKS$ / MKD$

```
MKtype$(number)
```

Encodes a number as a Latin-1 binary string.

Example

```
field$ = MKI$(7)
```

MOD

```
left MOD right
```

Returns the integer remainder after rounded integer operands.

Example

```
PRINT 7 MOD 4
```

MOUSE

```
MOUSE(mode)
```

Mode 0 returns button state; 1 returns X; other modes return Y.

Example

```
x = MOUSE(1)
```

Implementation note Not all historical modes are implemented.

NAME

```
NAME old$ AS new$
```

Renames or moves a host file.

Example

```
NAME "a" AS "b"
```

NEXT

```
NEXT [variable]
```

Ends a FOR loop.

Example

```
NEXT i
```

NOT

```
NOT value
```

Returns the bitwise complement.

Example

```
mask = NOT flags
```

OBJECT properties

```
OBJECT.X(id), OBJECT.Y(id), OBJECT.VX(id), ...
```

Gets or sets portable object state.

Example

```
OBJECT.X(1) = 20
```

Implementation note Rendering of OBJECT.SHAPE is not implemented.

ON ERROR

```
ON ERROR GOTO label | ON ERROR GOTO 0
```

Installs or removes the program error handler.

Example

```
ON ERROR GOTO Handler
```

ON events

```
ON TIMER(n)|MOUSE|MENU|COLLISION GOSUB label
```

Installs an event subroutine.

Example

```
ON TIMER(1) GOSUB Tick
```

Implementation note Events are polled between statements.

ON GOTO / GOSUB

```
ON expression GOTO|GOSUB target[, ...]
```

Selects a branch using a one-based expression.

Example

```
ON choice GOTO One,Two
```

OPEN

```
OPEN path$ FOR mode AS #channel | OPEN path$ AS #channel LEN = size
```

Opens sequential or random host files.

Example

```
OPEN "a.txt" FOR INPUT AS #1
```

OPTION BASE

```
OPTION BASE 0|1
```

Sets the lower bound for subsequently dimensioned arrays.

Example

```
OPTION BASE 1
```

OR

```
left OR right
```

Returns bitwise OR.

Example

```
flags = flags OR 4
```

PAINT

```
PAINT (x,y)[, color]
```

Fills a retained rectangle or oval containing the point.

Example

```
PAINT (50,50), "red"
```

Implementation note Not a general pixel flood fill.

PALETTE / PATTERN

```
PALETTE ... | PATTERN ...
```

Accepted graphics compatibility forms.

Example

```
PATTERN 1
```

Implementation note Currently no-op.

PEEK

```
PEEK(address) | PEEKW(address) | PEEKL(address)
```

Reads the sandboxed virtual memory map.

Example

```
PRINT PEEK(100)
```

POINT

```
POINT(x,y)
```

Compatibility pixel-query function.

Example

```
PRINT POINT(1,1)
```

Implementation note Currently returns zero.

POKE

```
POKE address, value | POKEW ... | POKEL ...
```

Writes the sandboxed virtual memory map.

Example

```
POKE 100,77
```

PRESET / PSET

```
PSET (x,y)[, color] | PRESET (x,y)[, color]
```

Draws one pixel-sized rectangle on the active canvas.

Example

```
PSET (10,10)
```

PRINT

`PRINT [expression][,|; ...]`

Writes formatted text to output.

Example

```
PRINT "Value"; value
```

PRINT USING

`PRINT USING pattern$; values`

Formats common numeric # fields.

Example

```
PRINT USING "###.##"; price
```

PROCESS RUN

`PROCESS RUN path$[, argument$...]`

Starts a host process asynchronously.

Example

```
PROCESS RUN "/usr/bin/open", "."
```

PTAB / SPC / TAB

`function(count)`

Returns count spaces for formatted output.

Example

```
PRINT TAB(8); name$
```

Implementation note Does not inspect the true cursor column.

PSET

See `PRESET / PSET`

Sets a point on the active canvas.

Example

```
PSET (1,1)
```

PUT

`PUT #channel[, record]`

Writes FIELD variables to a random-access record.

Example

```
PUT #2,1
```

RANDOMIZE

```
RANDOMIZE [seed]
```

Seeds the pseudorandom generator.

Example

```
RANDOMIZE 123
```

READ

```
READ variable[, ...]
```

Consumes values from the DATA store.

Example

```
READ name$,score
```

REM

```
REM comment
```

Begins a comment.

Example

```
REM setup
```

RESTORE

```
RESTORE [label]
```

Moves the DATA pointer.

Example

```
RESTORE Names
```

RESUME

```
RESUME | RESUME NEXT | RESUME label
```

Continues after a trapped error.

Example

```
RESUME NEXT
```

RETURN

```
RETURN [expression]
```

Returns from GOSUB or a structured procedure.

Example

```
RETURN total
```

RIGHT$

```
RIGHT$(text$, count)
```

Returns the rightmost characters.

Example

```
PRINT RIGHT$("MacBasic",5)
```

RND

```
RND | RND()
```

Returns a pseudorandom value between zero and one.

Example

```
PRINT RND
```

RUN

```
RUN path$
```

Loads and runs a source file.

Example

```
RUN "demo.bas"
```

SAVE

```
SAVE path$
```

Writes prepared source lines to a file.

Example

```
SAVE "copy.bas"
```

Implementation note Numeric line labels are not preserved.

SAY

```
SAY text$
```

Speaks text through the host speech command.

Example

```
SAY "Hello"
```

SCREEN

```
SCREEN id, width, height[, ...]
```

Creates a portable screen window and canvas.

Example

```
SCREEN 1,640,400
```

Implementation note Depth and chipset display modes are ignored.

SCROLL / WIDTH

```
SCROLL ... | WIDTH ...
```

Accepted graphics compatibility forms.

Example

```
WIDTH 2
```

Implementation note Currently no-op.

SGN

```
SGN(number)
```

Returns -1, 0, or 1 according to sign.

Example

```
PRINT SGN(-2)
```

SIN

```
SIN(number)
```

Returns the sine of radians.

Example

```
PRINT SIN(angle)
```

SLEEP

```
SLEEP seconds
```

Pauses the interpreter thread.

Example

```
SLEEP .25
```

Implementation note Events do not interrupt SLEEP.

SOUND

```
SOUND frequency, duration[, volume[, voice]]
```

Synthesizes a tone.

Example

```
SOUND 440,.25,.8,0
```

SOUND PLAY / STOP

```
SOUND PLAY nameOrPath$ | SOUND STOP
```

Plays named/file audio or stops program-started sounds.

Example

```
SOUND PLAY "Glass"
```

SPACE$

```
SPACE$(count)
```

Returns a string of spaces.

Example

```
padding$ = SPACE$(10)
```

SQR

```
SQR(number)
```

Returns the square root.

Example

```
PRINT SQR(16)
```

STATIC / SHARED / COMMON / DECLARE

```
compatibility declaration
```

Accepted for source compatibility.

Example

```
SHARED total
```

Implementation note Currently no-op.

STEP

```
FOR ... STEP increment
```

Specifies a FOR-loop increment.

Example

```
FOR i=10 TO 1 STEP -1
```

STICK / STRIG

```
STICK(port) | STRIG(port)
```

Reserved controller queries.

Example

```
PRINT STICK(0)
```

Implementation note Currently return zero.

STOP

```
STOP
```

Ends program execution.

Example

```
STOP
```

STR$

```
STR$(value)
```

Returns the printable string representation.

Example

```
text$ = STR$(42)
```

STRING$

```
STRING$(count, character)
```

Repeats the first character.

Example

```
PRINT STRING$(5,"*")
```

SUB

```
SUB name(parameters) ... END SUB
```

Defines a structured subroutine.

Example

```
SUB Hello()
 PRINT "Hi"
END SUB
```

SWAP

```
SWAP variable1, variable2
```

Exchanges two values.

Example

```
SWAP a,b
```

SYSTEM / NEW

```
SYSTEM | NEW
```

Ends execution.

Example

```
SYSTEM
```

Implementation note NEW does not clear the IDE document.

TAN

```
TAN(number)
```

Returns the tangent of radians.

Example

```
PRINT TAN(angle)
```

TIME$

```
TIME$
```

Returns current time as HH:mm:ss.

Example

```
PRINT TIME$
```

TIMER

```
TIMER | TIMER ON|OFF|STOP
```

Returns epoch seconds or controls an ON TIMER trap.

Example

```
PRINT TIMER
```

Implementation note The value differs from historical Amiga semantics.

TO

```
FOR ... TO limit
```

Introduces the inclusive FOR-loop limit.

Example

```
FOR i=1 TO 5
```

TRANSLATE$

```
TRANSLATE$(text$)
```

Returns text unchanged as a compatibility placeholder.

Example

```
PRINT TRANSLATE$(text$)
```

Implementation note No character translation table is applied.

UBOUND

```
UBOUND(array[, dimension])
```

Returns the upper bound; dimensions are numbered from one.

Example

```
PRINT UBOUND(board,2)
```

UCASE$ / LCASE$

```
UCASE$(text$) | LCASE$(text$)
```

Changes string case.

Example

```
PRINT UCASE$("Mac")
```

VAL

```
VAL(text$)
```

Converts leading numeric text to a number.

Example

```
PRINT VAL("42")
```

VARPTR / SADD

```
VARPTR(value) | SADD(value)
```

Returns a host-related compatibility value.

Example

```
PRINT VARPTR(a)
```

Implementation note Not a stable storage address.

VIEW ADD

```
VIEW ADD viewId, windowId, x, y, width, height
```

Creates an ID-addressable drawing canvas in the window selected by ID.

Example

```
VIEW ADD 10,1,0,0,400,300
```

WAIT

```
WAIT address, mask
```

Waits until virtual memory at address has a masked bit.

Example

```
WAIT 100,64
```

WAVE

```
WAVE voice, waveform
```

Selects sine (0), square (1), or sawtooth (2).

Example

```
WAVE 0,1
```

WEND

```
WEND
```

Ends a WHILE loop.

Example

```
WEND
```

WHILE

```
WHILE condition
```

Begins a condition-controlled loop.

Example

```
WHILE running
```

WINDOW

```
WINDOW OPEN windowId, title$, w, h[, x, y] | WINDOW CLOSE windowId
```

Creates or closes an ID-addressable native host window. The title is display text, not the routing key.

Example

```
WINDOW OPEN 1,"Demo",640,400
```

WRITE

```
WRITE #channel, value[, ...]
```

Writes comma-delimited values, quoting strings.

Example

```
WRITE #1,"Ada",42
```

XOR

```
left XOR right
```

Returns bitwise exclusive OR.

Example

```
result = a XOR b
```

CHAPTER 14

Complete Examples

Programs that combine the language facilities

Graphics Demonstration

```
' MacBasic drawing example
' String literals use the straight ASCII U+0022 delimiter only.
WINDOW OPEN 1, "Drawing Demo", 640, 420, 160, 140
VIEW ADD 10, 1, 20, 20, 600, 360

CLEAR 10, "#F5F3FF"
DRAW RECT 10, 30, 30, 540, 300, "#4F46E5", 0
DRAW OVAL 10, 70, 80, 160, 160, "#06B6D4", 1
DRAW OVAL 10, 370, 80, 160, 160, "#EC4899", 1
DRAW LINE 10, 230, 160, 370, 160, "#111827"
DRAW TEXT 10, "MacBasic Graphics", 205, 280, "#312E81"
```

Procedures, Arrays, and DATA

```
OPTION BASE 1
DIM scores(3)
Names:
DATA "Ada", "Grace", "Linus"

FUNCTION Grade(score)
  IF score >= 90 THEN
    RETURN "A"
  ELSEIF score >= 80 THEN
    RETURN "B"
  ELSE
    RETURN "C"
  END IF
END FUNCTION

FOR i = 1 TO 3
  READ name$
  scores(i) = 75 + i * 6
  PRINT name$, scores(i), Grade(scores(i))
NEXT i
```

File Report

```
OPEN "scores.csv" FOR OUTPUT AS #1
WRITE #1, "Name", "Score"
FOR i = 1 TO 3
  WRITE #1, names$(i), scores(i)
NEXT i
CLOSE #1
```

CHAPTER 15

Keyword Index

Quick lookup by language area

Area	Keywords
Program structure	SUB, FUNCTION, RETURN, END, STOP, GOTO, GOSUB, ON, IF, ELSEIF, ELSE, FOR, NEXT, WHILE, WEND, EXIT
Variables and arrays	LET, DIM, ERASE, OPTION BASE, LBOUND, UBOUND, SWAP, DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR
Stored data	DATA, READ, RESTORE
Text I/O	PRINT, PRINT USING, INPUT, LINE INPUT, INPUT$, WRITE
Files	OPEN, CLOSE, FIELD, GET, PUT, LSET, RSET, EOF, LOC, LOF, FILES, CHDIR, KILL, NAME
Graphics	WINDOW, VIEW ADD, CLEAR, DRAW LINE, DRAW RECT, DRAW OVAL, DRAW TEXT, COLOR, PSET, PRESET, LINE, CIRCLE, AREA, AREAFILL, PAINT, SCREEN
Sound and speech	BEEP, SOUND, SOUND PLAY, SOUND STOP, WAVE, SAY
Events	TIMER, MOUSE, MENU, COLLISION, INKEY$, ON TIMER, ON MOUSE, ON MENU, ON COLLISION
System	PROCESS RUN, SLEEP, CLS, LIST, SAVE, CHAIN, RUN, SYSTEM, PEEK, POKE, WAIT

Reserved-Word Policy

MacBasic recognizes keywords by context and does not maintain a separate global reserved-word ban. Avoid using documented keywords as variable or procedure names because future versions may reserve them more strictly.

MacBasic

Structured BASIC for the Mac and GNUstep

Documentation generated from the MacBasic 0.2 implementation.